

FaceShape

Personajes SVG animados para React

Nombre de trabajo: FaceShape. El nombre definitivo del paquete npm queda pendiente.

1. Objetivo

Crear una librería que permita convertir una forma SVG en un personaje, agregándole una cara configurable, expresiones e interacción. La forma puede ser una de las incluidas o una forma propia definida por quien usa la librería.

La misma cara debe adaptarse a distintas siluetas mediante una región de posicionamiento llamada faceBox. Las animaciones forman parte del diseño inicial y pueden combinarse.

```
<Character
  shape="blob"
  expression="happy"
  color="#388697"
  size={200}
  motion={{ idle: true, blink: true, lookAt: "cursor" }}
/>
```

2. Principios acordados

- Separar forma, cara, expresión y movimiento.
- Usar SVG y geometría parametrizada para poder interpolar expresiones.
- Mantener el motor independiente de React y publicar inicialmente un solo paquete.
- Evitar una dependencia obligatoria de una librería de animación externa.
- Excluir audio del alcance inicial; talking es únicamente una animación visual.

Este documento consolida lo conversado. Los tipos y contratos detallados que siguen son una propuesta inicial para implementar, todavía sujeta a validación durante el desarrollo.

3. Alcance de la primera versión

Área	Incluido en v1
Formas	Círculo, blob, cuadrado redondeado y estrella; soporte de SVG/path propio.
Cara	Ojos, pupilas, boca y cejas independientes. Variantes y composición manual.
Expresiones	neutral, happy, sad, angry, surprised y sleepy.
Movimiento	idle, blink, bounce, shake y talking visual. Transiciones suaves entre expresiones.
Interacción	Seguimiento del cursor y dirección de mirada controlada mediante coordenadas.
Personalización	Color, tamaño, faceBox, variantes, seed, className y style.
Accesibilidad	Respeto por prefers-reduced-motion y posibilidad de etiquetar o marcar el SVG como decorativo.

Fuera del alcance inicial

- Micrófono, análisis de amplitud, sincronización labial y cualquier integración de audio.
- Accesorios como anteojos y un sistema avanzado de animaciones registrables.
- Detección automática de la región de la cara en un SVG arbitrario.

Extensibilidad prevista

El diseño debe permitir incorporar formas y expresiones propias. El registro de animaciones personalizadas queda como evolución posterior. La cantidad exacta de variantes de ojos y boca se definirá al construir los primeros presets.

Dificultad principal

El desafío está en mantener una cara proporcionada sobre siluetas distintas y en lograr transiciones naturales. El componente React funciona como una capa de integración sobre ese motor geométrico.

4. Arquitectura y modelo geométrico

Capa	Responsabilidad
Shape	Dibujar la silueta y declarar su faceBox.
Face	Combinar ojos, pupilas, boca y cejas.
Expression	Describir los parámetros visuales de una emoción.
Motion	Aplicar movimiento y comportamiento temporal sin cambiar la identidad de la expresión.

Separación interna

```
src/  
  core/  
    geometry/      # geometría y coordenadas  
    shapes/        # formas y regiones de cara  
    expressions/   # presets y parámetros  
    motion/        # estado de movimiento  
    interpolation/  # transición entre estados  
  react/  
    Character, Face, Eyes, Mouth, Eyebrows  
    hooks/
```

Los nombres `@faceshape/core` y `@faceshape/react` describen una posible separación futura. La v1 puede exponer todo desde un único paquete para mantener simple la instalación.

Coordenadas y faceBox

La cara utiliza coordenadas normalizadas de 0 a 1. Cada forma declara `x`, `y`, `width` y `height` de su `faceBox`, también normalizados respecto de su `viewBox`. La transformación ubica los rasgos dentro de esa región.

```
type FaceBox = {  
  x: number;  
  y: number;  
  width: number;  
  height: number;  
};  
  
// Ejemplo de región en una forma:  
const faceBox = { x: 0.25, y: 0.25,  
                  width: 0.5, height: 0.45 };
```

Para formas irregulares, quien define el SVG ajusta la región manualmente. Un SVG propio requiere un `viewBox` conocido; el sistema no interpreta cualquier archivo SVG ni calcula automáticamente un área segura.

5. Expresiones y animaciones

Expresión describe cómo se ve la cara. Movimiento describe qué hace. Por ejemplo, angry puede combinarse con bounce, y happy con idle y blink simultáneamente.

Geometría parametrizada

Los presets se resuelven a valores numéricos: apertura de ojos, posición de pupilas, curvatura y apertura de boca, ángulo de cejas, entre otros. Los paths de la cara se generan a partir de esos valores, en lugar de depender de dibujos independientes para cada emoción.

```
// Modelo conceptual del motor, no contrato final:
type MouthGeometry = {
  curve: number;
  openness: number;
  width: number;
};

type MotionConfig = {
  idle?: boolean;
  blink?: boolean;
  bounce?: boolean;
  shake?: boolean;
  talking?: boolean;
  lookAt?: "cursor" | { x: number; y: number };
};
```

Composición y transiciones

motion es un objeto para permitir movimientos simultáneos. Conviene aplicar transformaciones en grupos SVG separados: movimiento global del cuerpo, parpadeo de ojos, mirada de pupilas y movimiento de boca. Esto evita que una animación sobrescriba a otra.

Al cambiar expression, el motor interpola desde los valores visibles hasta los del nuevo preset. Si llega otro cambio durante la transición, debe partir del estado actual. La animación talking modifica la apertura de la boca sobre la expresión base.

```
<Character
  expression={isError ? "sad" : "happy"}
  motion={{ idle: true, blink: true, talking: isTalking }}
  transition={{ duration: 300, easing: "ease-out" }}
/>
```

Como contrato propuesto, lookAt con coordenadas usa el rango 0 a 1: { x: 0.5, y: 0.5 } representa la mirada al centro. El motor limita el desplazamiento para mantener las pupilas dentro de los ojos.

6. API pública propuesta

Forma propia y composición manual

```
<Character
  shape={{
    path: mySVGPath,
    viewBox: "0 0 100 100",
    faceBox: { x: 0.2, y: 0.25, width: 0.6, height: 0.5 }
  }}
  expression="happy"
  face={{ eyes: "round", mouth: "smile" }}
  seed="ezequiel"
  size={200}
/>

// Composición avanzada prevista:
<Character shape={shape}>
  <Eyes variant="cute" />
  <Mouth variant="smile" />
  <Eyebrows variant="soft" />
</Character>
```

Los nombres de variantes del ejemplo son ilustrativos. La precedencia propuesta es: preset de expresión, luego ajustes explícitos de face. Los componentes hijos reemplazan las partes equivalentes; este contrato debe cerrarse antes de implementar la API avanzada.

Contratos a cerrar durante la implementación

- Formato para formas con varios paths o un componente SVG propio, además del path simple.
- Parámetros, rangos y variantes definitivas de ojos, boca y cejas.
- Comportamiento repetido o puntual de bounce y shake, y mecanismo para volver a dispararlos.
- Qué variaciones modifica seed: mantener la misma semilla reproducible y no alterar opciones explícitas.

7. Implementación y validación

Orden sugerido: definir tipos y contratos; construir geometría y presets estáticos; agregar interpolación y movimientos; integrar React; preparar ejemplos, documentación y empaquetado npm.

La validación debe cubrir la misma expresión en todas las formas, una forma propia, cambios de emoción durante una transición, movimientos combinados y resultados consistentes con seed. También debe revisar reduced motion, limpieza de listeners y animaciones al desmontar, y uso sin acceso al navegador durante el render inicial.

Resultado esperado: una primera versión reutilizable, con formas propias, caras configurables y animaciones combinables, lista para documentar y preparar para publicación en npm.